

DATA 260 HOMEWORK 1

Name: Ayushi Shringi

SID4: 0036

DOMAIN_ID: 4

Domain Entity: Open-source package vulnerabilities

Port: 8036

Prefix: s0036

SEED: 0036

VERIFY_SEED: 260036

Python Version: Python 3.12.14

Model: qwen3:4b

Hardware: MacBook Pro

Chip: Apple M5 Pro

Memory: 24 GB

Operating System: macOS Tahoe 26.5.1

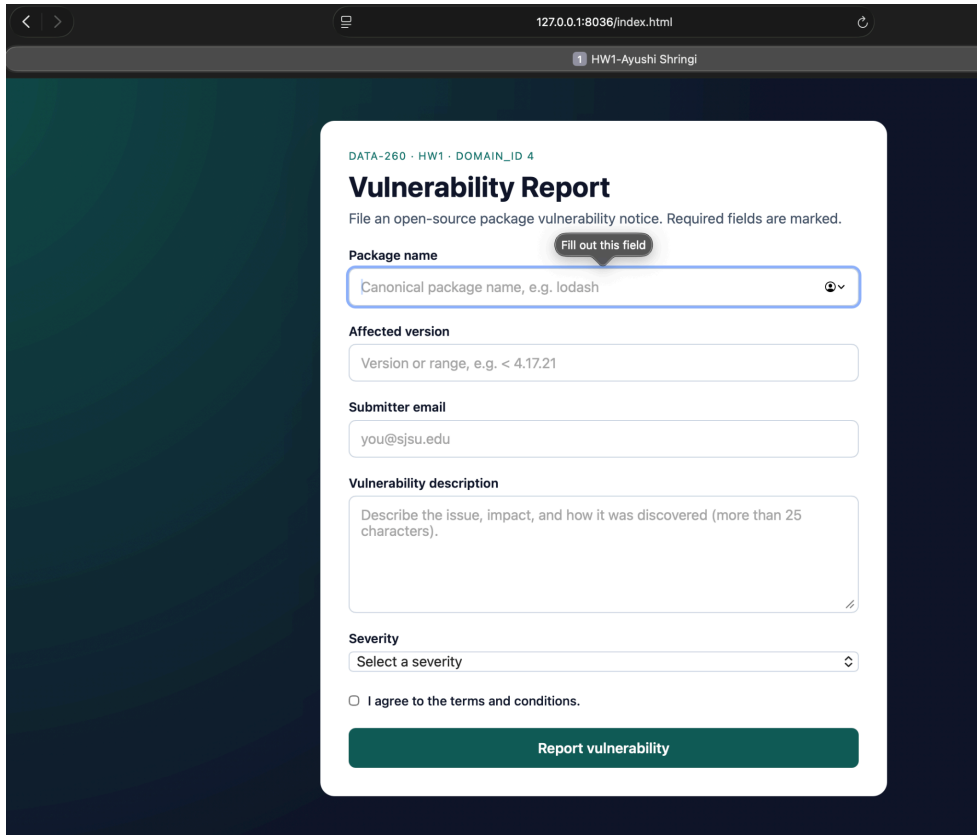
GitHub Repository: <https://github.com/ayushishringi/data260-0036>

Commit Hash: c64e34015b42da54bf77657e7ddf63a581509555

Git Tag: hw1

Part I and Part II: HTML and JavaScript Form

Required-field validation -



DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name Fill out this field

Canonical package name, e.g. lodash

Affected version

Version or range, e.g. < 4.17.21

Submitter email

you@sjsu.edu

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

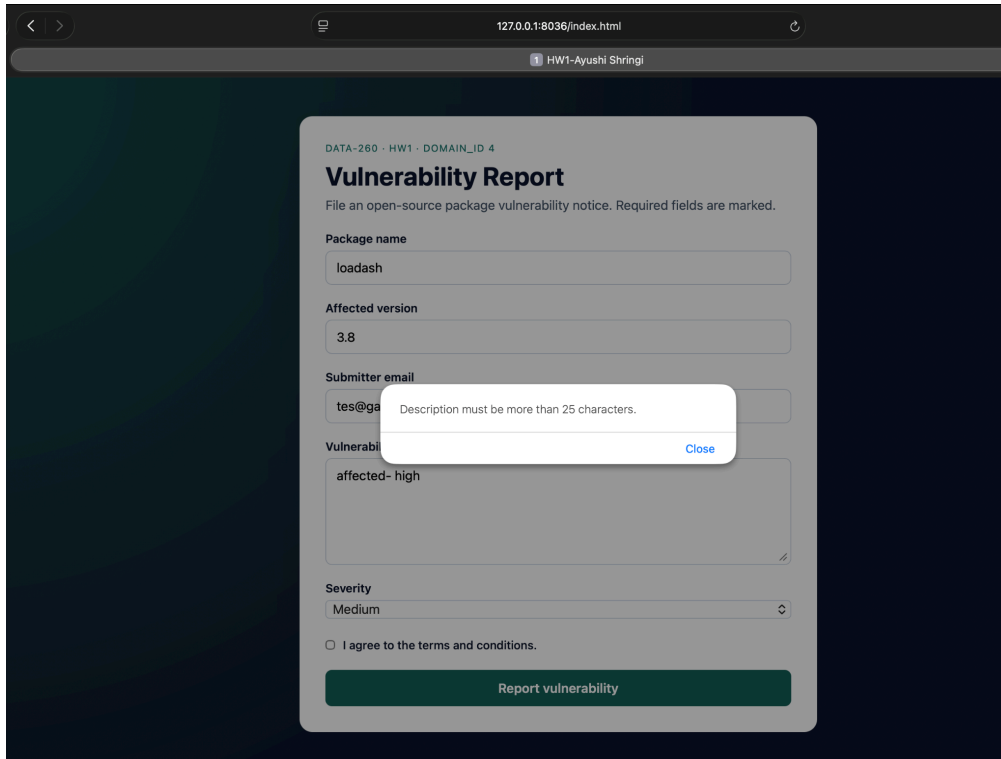
Severity

Select a severity

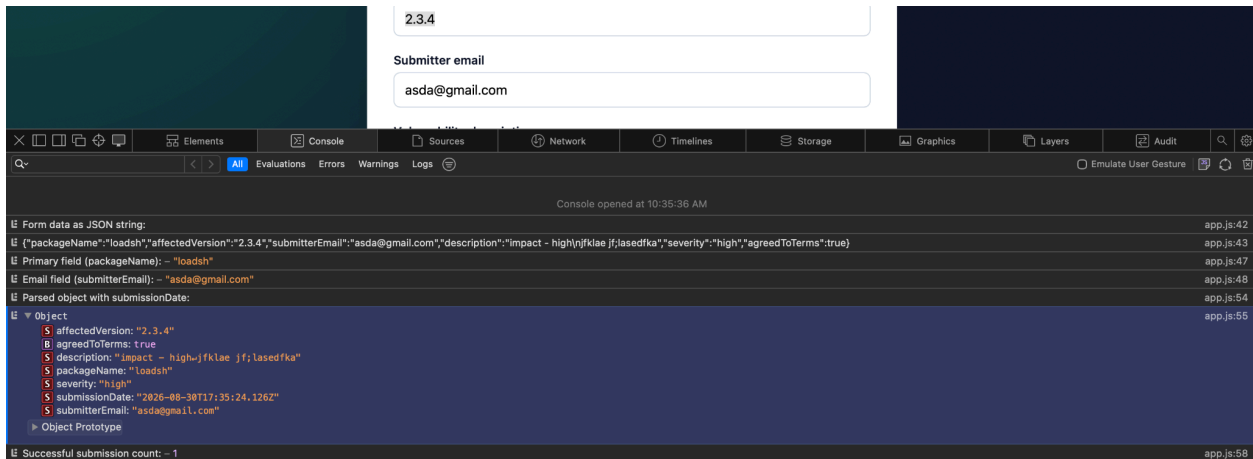
☐ I agree to the terms and conditions.

Report vulnerability

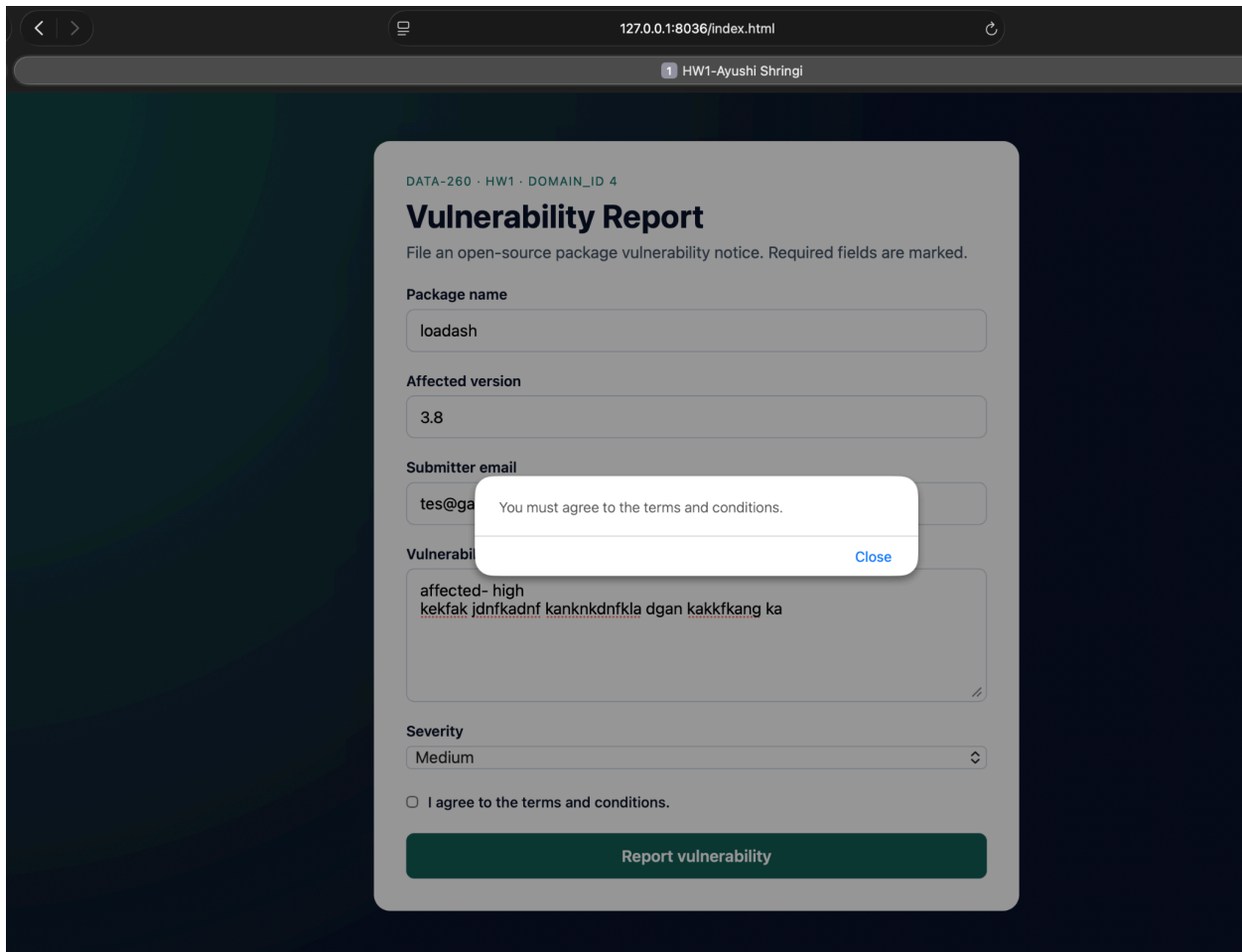
Description Length Verification-



Console Logging-

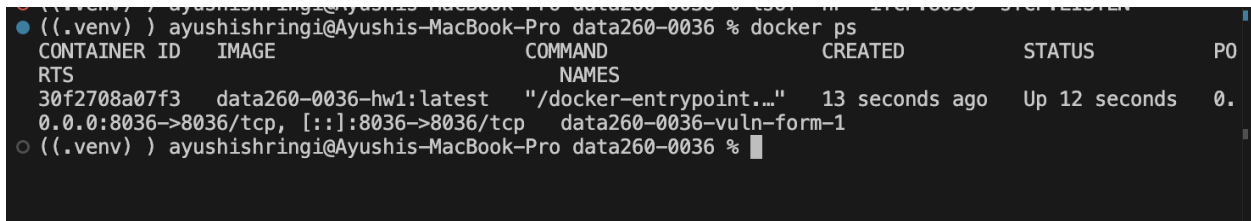


Terms-and-Conditions Validation -



Part III: Docker Containerization

Docker container running locally-



From running through Docker-

127.0.0.1:8036/index.html

HW1-Ayushi Shrangi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name

Canonical package name, e.g. lodash

Affected version

Version or range, e.g. < 4.17.21

Submitter email

you@sjsu.edu

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

Severity

Select a severity

☐ I agree to the terms and conditions.

Report vulnerability

Console Logging-

2.3.4

Submitter email

asda@gmail.com

Elements

Console

Sources

Network

Timelines

Storage

Graphics

Layers

Audit

Emulate User Gesture

Console opened at 10:35:36 AM

Form data as JSON string:

app.js:42

("packageName":"lodash","affectedVersion":"2.3.4","submitterEmail":"asda@gmail.com","description":"impact - high,jfklae jf,lasedfka","severity":"high","agreedToTerms":true)

app.js:43

Primary field (packageName): - "lodash"

app.js:47

Email field (submitterEmail): - "asda@gmail.com"

app.js:48

Parsed object with submissionDate:

app.js:54

Object

app.js:55

affectedVersion: "2.3.4"

agreedToTerms: true

description: "impact - high,jfklae jf,lasedfka"

packageName: "lodash"

severity: "high"

submissionDate: "2026-08-30T17:35:24.126Z"

submitterEmail: "asda@gmail.com"

Object Prototype

Successful submission count: - 1

app.js:58

ECS Service-

Clusters > data260-0036-cluster > Services

data260-0036-service has been deployed successfully. [View service](#)

data260-0036-cluster Last updated August 30, 2026, 14:13 (UTC-7:00) [Actions](#) [Create with Express Mode](#)

Cluster overview

Cluster ARN: [arn:aws:ecs:us-west-2:232025405374:cluster/da...](#) **Status** Active

Services 0 active | 0 draining **Tasks** 0 running | 0 pending

CloudWatch monitoring Default **Action logs** Turned off

Registered container instances -

[Services](#) | [Tasks](#) | [Daemons](#) | [Infrastructure](#) | [Metrics](#) | [Scheduled tasks](#) | [Configuration - updated](#) | [Event history](#) | [Tags](#)

Services (1) [info](#) Last updated August 30, 2026, 14:30 (UTC-7:00) [Manage tags](#) [Update](#) [Delete service](#) [Create](#)

Filter services by value

Filter launch type: Any launch type

Filter scheduling strategy: Any scheduling strategy

Filter resource management type: Any resource management type

☐	Service name	ARN	Status	Schedu...	Launc...	Task de...	Deployments and tasks	Last deployment	Last deplo
☐	data260-0036-service	arn:aws:ec...	Active	Replica	Fargate	data260-...	1/1 Tasks running	Completed	View 17 minutes

Container Service > Clusters > data260-0036-cluster > Services > data260-0036-service > Tasks > 24923e973a4e4d74bc5e8029c4cab2e5 > Configuration

Configuration

Operating system/Architecture: Linux/ARM64

Capacity provider: -

Platform version: 1.4.0

Task role: -

Task group: [service:data260-0036-service](#) [View service](#)

ECS Exec: [Info](#) Turned off

Task execution role: [ecsTaskExecutionRole](#) [L2](#)

Fault injection: Turned off

Fargate ephemeral storage

Encryption: [Info](#) Default AWS Fargate encryption

Size (GiB): 20

Networking

Network mode: awsvpc

ENI ID: [eni-0f56eac57276f9e65](#) [L2](#)

Subnet: [subnet-04d28f0566eace32](#) [L2](#)

Private IP: [172.31.16.122](#)

Public IP: [44.247.157.122](#) [open address](#) [L2](#)

IPv6 address: -

Security group: [sg-08b29299d412da2c5](#) ([data260-0036-sg](#)) [L2](#)

MAC address: [02:ff:daa7:d9:5d](#)

[Run Reachability Analyzer](#) [L2](#)

Task scale-in protection

Protection status: Turned off

Protection duration: -

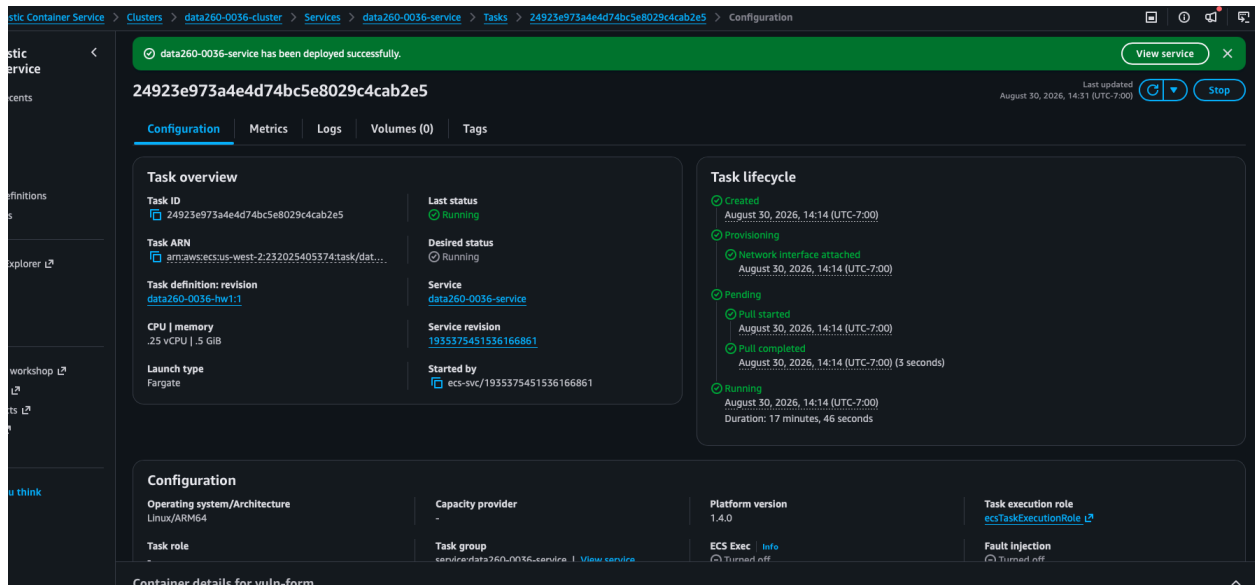
[Configure task scale-in protection](#)

Containers (1) [Connect](#)

Filter containers

Container name	Container runtime ID	Image URI	Image Digest	Status	Health status	CPU	Memory hard/soft limit
vuln-form	24923e973a4e4d...	docker.io/...	sha256:f8...	Running	Unknown	0	- / -

Task Details-



Application Deployed on AWS-

Task configuration | Elastic Container Service | us-west-2

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name

Affected version

Submitter email

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

Severity

Select a severity

☐ I agree to the terms and conditions.

Report vulnerability

Planner–Reviewer–Finalizer agent pipeline-

The screenshot shows a VS Code editor with a file explorer on the left containing files like .gitignore, AGENT.md, agents_demo.py, app.js, docker-compose.yml, Dockerfile, DOMAIN_SCHEMA.md, hw1_client.py, index.html, Makefile, nginx.conf, README.md, and styles.css. The main editor displays a Python script with the following code:

```
168         "notes": "Reviewer JSON missing tags; kept planner output.",
169     }
170     planner_tags = planner_json.get("tags", [])
171     planner_summary = str(planner_json.get("summary", ""))
172     reviewer_summary = str(reviewer_json.get("summary", ""))
173
174     # Preserve a valid Planner summary exactly when it is already within
175     # the assignment's 25-word limit.
```

The terminal output shows the command `python3 agents_demo.py --input reports/hw01/cases/nondeterminism_input.json` and its output, which includes a JSON structure for the planner and reviewer outputs, and a final summary of the results.

Agentic AI Pipeline

Forty-run nondeterminism experiment -

The screenshot shows a VS Code editor with a file explorer on the left containing files like .gitignore, AGENT.md, agents_demo.py, app.js, docker-compose.yml, Dockerfile, DOMAIN_SCHEMA.md, hw1_client.py, index.html, Makefile, nginx.conf, README.md, and styles.css. The main editor displays a Python script with the following code:

```
168         "notes": "Reviewer JSON missing tags; kept planner output.",
169     }
170     planner_tags = planner_json.get("tags", [])
171     planner_summary = str(planner_json.get("summary", ""))
172     reviewer_summary = str(reviewer_json.get("summary", ""))
173
174     # Preserve a valid Planner summary exactly when it is already within
175     # the assignment's 25-word limit.
```

The terminal output shows the command `python -c 'import csv,json,sys,collections; rows=list(csv.DictReader(open("reports/hw01/raw/nondeterminism_40_runs.csv"))); keysnext(k for k in rows[0] if k.lower() in ("temperature","temp")); counts=collections.Counter(str(r[key]) for r in rows); metrics=json.load(open("reports/hw01/raw/nondeterminism_metrics.json")); print("Python version:",sys.version.split()[0]); print("Total runs:",len(rows)); print("Temperature 0.0 runs:",counts["0.0"]); print("Temperature 0.7 runs:",counts["0.7"]); print("Temp 0.0 distinct tag sets:",metrics["0.0"]["distinct_tag_sets"]); print("Temp 0.7 distinct tag sets:",metrics["0.7"]["distinct_tag_sets"]); print("Result: temperature 0.7 produced greater output variability.")'` and its output, which includes a summary of the results.

Five-Turn Client and Token Accounting-


```

((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % python3 -u hw1_client.py | tee reports/
hw01/raw/hw1_client_console.txt

=== turn 1 user ===
Review this function: def add(a,b): return a+b
=== turn 1 assistant ===
- The function correctly computes the sum of two numeric inputs (if they are numbers).
- The function has no security vulnerabilities as it does not interact with external systems.
[tokens] input=106 output=3273 total=3379

=== turn 2 user ===
The function does not check types. What would you change?
=== turn 2 assistant ===
- Add type validation to ensure both a and b are numeric (e.g., integers or floats) before returning
their sum.
[tokens] input=161 output=1522 total=1683

=== turn 3 user ===
Rewrite the review for a version that adds logging of arguments.
=== turn 3 assistant ===
- The function correctly logs the arguments before returning their sum, improving debuggability.
- The function has no security vulnerabilities as it does not interact with external systems.
[tokens] input=209 output=4275 total=4484
--- /stats ---
turn_count: 3
cumulative_input_tokens: 476
cumulative_output_tokens: 9070
cumulative_total_tokens: 9546
serialized_conversation_history_length: 1272
=====

=== turn 4 user ===
Is logging user-controlled strings a security issue here?
=== turn 4 assistant ===
- Logging user-controlled strings can expose sensitive information if the logs are accessible to unau-
thorized users, posing a security risk.
- The function does not introduce new security vulnerabilities beyond the logging mechanism itself.
[tokens] input=262 output=3173 total=3435

=== turn 5 user ===
Give a final bullet-only verdict on shipping add().
=== turn 5 assistant ===
- The function should not be shipped as logging user-controlled strings could expose sensitive inform-
ation if logs are accessible to unauthorized users.
[tokens] input=321 output=1076 total=1397
--- /stats ---
turn_count: 5
cumulative_input_tokens: 1059
cumulative_output_tokens: 13319
cumulative_total_tokens: 14378
serialized_conversation_history_length: 1917
=====

=== exit totals ===
cumulative_input_tokens: 1059
cumulative_output_tokens: 13319
cumulative_total_tokens: 14378
turn_count: 5
machine_readable_tokens: reports/hw01/raw/hw1_client_tokens.json

```

Machine-readable five-turn client verification-

```

• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % python3 -c 'import json; d=json.load(open("reports/hw01/raw/hw1_client_tokens.json")); bullet_ok=all(all(line.strip().startswith("- ") for line in t["assistant"].splitlines() if line.strip()) for t in d["turns"]); print("Python:",d["python_version"]); print("Model:",d["model"]); print("Turns:",len(d["turns"])); print("Stats after turns:",[s["after_turn"] for s in d["stats"]]); print("Totals:",d["totals"]); print("Bullet-only compliance:",bullet_ok)'
Python: 3.12.14
Model: qwen3:4b
Turns: 5
Stats after turns: [3, 5]
Totals: {'turn_count': 5, 'cumulative_input_tokens': 1059, 'cumulative_output_tokens': 13319, 'cumulative_total_tokens': 14378}
Bullet-only compliance: True

```

Final automated verification-

```

Problems 2 Output Debug Console Terminal Ports
• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % python3 scripts/verify_hw01.py
{
  "passed": true,
  "failed_files": [],
  "checks": {
    "python_version": "3.12.14",
    "python_major_minor": "3.12",
    "config": {
      "SID4": "0036",
      "PORT_BASE": 8036,
      "PREFIX": "s0036",
      "SEED": "0036",
      "VERIFY_SEED": 260036,
      "DOMAIN_ID": 4,
      "assigned_domain": "Open-source package vulnerabilities"
    },
    "files": {
      "index.html": true,
      "app.js": true,
      "styles.css": true,
      "DOMAIN_SCHEMA.md": true,
      "AGENT.md": true,
      "agents_demo.py": true,
      "hw1_client.py": true,
      "src/model_client.py": true,
      "Dockerfile": true,
      "docker-compose.yml": true,
      "Makefile": true,
      "README.md": true,
      "reports/hw01/cases/nondeterminism_input.json": true,
      "reports/hw01/AI_USE.md": true,
      "reports/hw01/METRICS.md": true
    },
    "html_title": true,
    "h1_entity": true,
    "schema_matches_domain": true,
    "localhost_port_open": false,
    "ollama_reachable": true,
    "ollama_models": "qwen3:4b",
    "timestamp_utc": "2026-08-30T23:58:04.605887+00:00"
  }
}

```

AI Use - HW1

```

1. **What AI assistants were used for versus what I did.**

```

I used ChatGPT/Codex to help draft and review the Ollama model adapter, Planner-Reviewer-Finalizer pipeline, verification scripts, and report structure. I reviewed the generated code, executed every command, tested the form, ran Docker and Ollama locally, completed the 40-run nondeterminism experiment, and collected the report evidence.

2. ****AI-produced output that was wrong or unsuitable.****

The Qwen3 Reviewer sometimes incorrectly changed the valid version range ``3.21-3.25`` to forms such as ``3.21-3:25``. Additionally, Ollama sometimes returned internal reasoning text even though the request specified ``think: false``.

3. ****How the problems were detected and verified.****

I detected the version-range error by comparing the Planner, Reviewer, and Finalizer console outputs. I detected the reasoning-text problem through a direct Ollama API test and unusually large output-token counts. I reran the agent pipeline and five-turn client after making corrections and verified the saved machine-readable results.

4. ****What changed and why it works now.****

I added deterministic validation that preserves a valid Planner summary when it is already within the 25-word limit and accurately calculates the Reviewer's ``changed`` value. I also updated ``src/model_client.py`` to remove reasoning text ending in ``</think>`` before returning model content. The project now runs in a Python ``3.12.14`` virtual environment, produces clean bullet-only responses, records per-turn and cumulative token counts, and passes ``scripts/verify_hw01.py``.

Short Answers - Agentic Pipeline

Q1. The 3 final tags:

grpc-security, protobuf-vulnerability, network-privilege-escalation

Q2. The final summary(22 words):

"gRPC reflection in protobuf-java 3.21-3.25 leaks private field names across package versions, enabling attackers to map internal authorization flags via error payloads."

Q3. Did the Reviewer agent change anything? Yes.

It shortened the summary from 28 to 22 words. It also briefly mangled the version range to "3.21-3:25" in its own draft output - the exact Reviewer quirk documented in AI_USE.md - but the Finalizer restored the correct

"3.21-3.25" in the published JSON.

Explanation of each step (in my own words):

- Planner reads the raw title/body cold and invents 3 tags plus a draft summary, with no awareness of any prior turn.
- Reviewer receives the original text plus the Planner's JSON, checks each tag actually appears as a concept in the source, verifies the summary is at most 25 words and factually unchanged (especially version numbers), and sets changed=true/false accordingly.
- Finalizer reconciles Planner + Reviewer into one publish JSON (exactly 3 tags, ≤25-word summary), preferring the Reviewer's values when valid, This is what caught and fixed the Reviewer's version-range typo in this run.

Part 4 - Conceptual Questions

Why is prior conversation context resent with every turn?

Stateless chat APIs do not store your session. Each complete() call sends the full messages list so the model can see prior turns.

How is a system prompt different from a user message?

A system prompt is a standing instruction (here: bullet-only code review, from AGENT.md). A user message is the current request in the conversation.

Why do input tokens grow over a conversation?

Because every later turn resends all earlier messages input tokens for turn N include the entire history up through turn N-1, not just the new question.

What eventually limits that growth?

The model's context window. Beyond that, any truncation or summarization strategy added in the client (this implementation does neither, so a long enough conversation would eventually exceed qwen3:4b's context limit and fail or get silently truncated by Ollama).